实验二 逻辑回归与朴素贝叶斯分类

一、实验目的

通过实验更好地掌握机器学习及优化的相关概念、技术、原理、应用等；通过实验提高学生编写实验报告、总结实验结果的能力；使对机器学习模型、优化算法等有比较深入的认识。

需掌握的知识点：

（1）掌握机器学习及优化中涉及的相关概念、模型、算法；

（2）熟悉机器学习模型训练、验证、测试的流程；

（3）熟悉常用的数据预处理方法；

（4）掌握逻辑回归、贝叶斯分类的表示、求解及编程。

二、基本要求

（1）准备实验数据，编程完成实验内容，收集实验结果。

（2）独立完成实验报告。

三、实验软件

推荐使用 Python 编程语言（允许使用 numpy 库，需实现详细实验步骤，不允许直接调用 scikit-learn 中回归、分类等高层 API）。

四、实验内容

基于 IRIS 鸢尾花数据集，完成样本是否属于维吉尼亚鸢尾(Iris-Virginica)的逻辑回归分类、朴素贝叶斯模型训练、测试与评估。

（1）准备数据集并认识数据

下载 IRIS 数据集

https://archive.ics.uci.edu/ml/datasets/iris

了解数据集各个维度特征及预测值的含义

（2）探索数据并预处理数据

观察数据集各个维度特征及预测值的数值类型与分布

（3）训练模型

编程实现训练数据集上逻辑回归模型的梯度下降参数求解、朴素贝叶斯参数统计

（4）测试和评估模型

在测试数据集上计算所训练模型的准确率、F1-score 等指标

五、学生实验报告要求

实验报告需要包含以下几个部分

（1）简要介绍逻辑回归分类的原理

给定数据集$D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_\mathrm{N}, y_\mathrm{N})\}$，其中$\boldsymbol{x}_i = (x_{i1}; x_{i2}; \cdots x_{id})$，$y_\mathrm{i} \in$

$\mathbb{R}$，$\boldsymbol{\theta} = (\theta_0; \theta_1; \theta_2; \cdots \theta_d)$，

Sigmoid 激活函数：将线性回归的输出映射到概率值(0, 1)区间。

$$\sigma(z)=\frac{1}{1+\mathrm{e}^{-\boldsymbol{z}}}$$

因此

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}_i)=\frac{1}{1+\mathrm{e}^{-\boldsymbol{\theta}^{\mathrm{T}}\boldsymbol{x}_i}}$$

损失函数：评估模型的预测与实际标签之间的差异。在逻辑回归中，对损失函数取对数：

$$J(\boldsymbol{\theta})=-\frac{1}{N}\sum_{i=1}^{N}\left[y_{\mathrm{i}}\log\left(h_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)+(1-y_{\mathrm{i}})\log\left(1-h_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)\right]$$

梯度下降：优化$\boldsymbol{\theta}$，使得损失最小。

$$\boldsymbol{\theta}=\boldsymbol{\theta}-\eta\nabla J(\boldsymbol{\theta})$$

对于每个$\boldsymbol{\theta}_j$分量，有：

$$\boldsymbol{\theta}_j=\boldsymbol{\theta}_j-\eta\frac{1}{N}\sum_{i=1}^{N}(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}_i\right)-y_{\mathrm{i}})x_{ij}$$

用向量表示：

$$\boldsymbol{\theta}=\boldsymbol{\theta}+\eta\frac{1}{N}\mathrm{X}^{\mathrm{T}}(\boldsymbol{y}-\sigma(\mathrm{X}\boldsymbol{\theta}))$$

解释：梯度推导过程如下

$$J(\boldsymbol{\theta})=-\frac{1}{N}\sum_{i=1}^{N}\left[y_{\mathrm{i}}\log\left(h_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)+(1-y_{\mathrm{i}})\log\left(1-h_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\right)\right]$$

令$z_i=\boldsymbol{\theta}^{\mathrm{T}}\boldsymbol{x}_i$，利用链式求导法则：

$$\frac{\partial J}{\partial\boldsymbol{\theta}}=\frac{\partial J}{\partial h_{\boldsymbol{\theta}}(z_i)}\frac{\partial h_{\boldsymbol{\theta}}(z_i)}{\partial z_i}\frac{\partial z_i}{\partial\boldsymbol{\theta}}$$

$$\frac{\partial J}{\partial \mathrm{h}_{\boldsymbol{\theta}}(z_i)}=-\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y_{\mathrm{i}}}{\mathrm{h}_{\boldsymbol{\theta}}(z_i)}-\frac{1-y_{\mathrm{i}}}{1-\mathrm{h}_{\boldsymbol{\theta}}(z_i)}\right]$$

由于$\mathrm{h}_{\boldsymbol{\theta}}(z_i)=\frac{1}{1+\mathrm{e}^{-z_i}}=\frac{\mathrm{e}^{z_i}}{1+\mathrm{e}^{z_i}}$，则有

$$\frac{\partial \mathrm{h}_{\boldsymbol{\theta}}(z_i)}{\partial z_i}=\frac{\mathrm{e}^{z_i}(1+\mathrm{e}^{z_i})-\mathrm{e}^{z_i}\mathrm{e}^{z_i}}{(1+\mathrm{e}^{z_i})^2}=\frac{\mathrm{e}^{z_i}}{1+\mathrm{e}^{z_i}}-(\frac{\mathrm{e}^{z_i}}{1+\mathrm{e}^{z_i}})^2=\mathrm{h}_{\boldsymbol{\theta}}(z_i)-(\mathrm{h}_{\boldsymbol{\theta}}(z_i))^2$$
$$=\mathrm{h}_{\boldsymbol{\theta}}(z_i)(1-\mathrm{h}_{\boldsymbol{\theta}}(z_i))$$

由于，$z_i=\boldsymbol{\theta}^T\boldsymbol{x}_i$，则有

$$\frac{\partial z_i}{\partial\boldsymbol{\theta}}=\boldsymbol{x}_i$$

所以

$$\frac{\partial J}{\partial\boldsymbol{\theta}}=\frac{\partial J}{\partial \mathrm{h}_{\boldsymbol{\theta}}(z_i)}\frac{\partial \mathrm{h}_{\boldsymbol{\theta}}(z_i)}{\partial z_i}\frac{\partial z_i}{\partial\boldsymbol{\theta}}$$

$$=-\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y^{(i)}}{\mathrm{h}_{\boldsymbol{\theta}}(z_i)}-\frac{1-y^{(i)}}{1-\mathrm{h}_{\boldsymbol{\theta}}(z_i)}\right]\mathrm{h}_{\boldsymbol{\theta}}(z_i)(1-\mathrm{h}_{\boldsymbol{\theta}}(z_i))\boldsymbol{x}_i$$

$$=-\frac{1}{N}\sum_{i=1}^{N}(y_i-\mathrm{h}_{\boldsymbol{\theta}}(z_i))\boldsymbol{x}_i$$

$$=\frac{1}{N}\sum_{i=1}^{N}(\mathrm{h}_{\boldsymbol{\theta}}(\boldsymbol{x}_i)-y_i)\boldsymbol{x}_i$$

同理，

$$\frac{\partial J}{\partial b}=\frac{1}{N}\sum_{i=1}^{N}(\mathrm{h}_{\boldsymbol{\theta}}(\boldsymbol{x}_i)-y_i)$$

本实验只要求判断样本是否属于维吉尼亚鸢尾(Iris-Virginica)的逻辑回归分类。逻辑回归通过将输入特征的线性组合转化为概率值来判断样本属于某一类别的可能性。模型会将这种线性组合结果输入到一个 sigmoid 函数中，将输出值限制在 0 到 1 之间，表示分类概率。逻辑回归通过不断优化参数来最小化预测误差，从而提高分类准确性。

（2）简要介绍朴素贝叶斯分类的原理

1）贝叶斯定理：对于某个样本$X=(x_1,x_2,\cdots,x_d)$

估计它属于类别k的概率：

$$\mathrm{P}(C_k|X)=\frac{\mathrm{P}(\mathrm{X}|C_k)\mathrm{P}(C_k)}{\mathrm{P}(X)}$$

2）条件独立性假设：朴素贝叶斯假设特征之间相互独立

$$\mathrm{P}(\mathrm{X}|C_k)=\mathrm{P}(x_1,x_2,\cdots,x_n|C_k)=\prod_{i=1}^{d}\mathrm{P}(x_i|C_k)$$

3）高斯朴素贝叶斯参数估计：在连续特征的情况下，假设特征满足正态分布

$$\mathrm{P}(x_i|C_k)=\frac{1}{\sqrt{2\pi}\sigma_{k,i}}\mathrm{e}^{-\frac{(x_i-\mu_{k,i})^2}{2\sigma_{k,i}^2}}$$

其中，$\mu_{k,i}$为类别C_k下特征x_i下的均值。

4）参数估计

$$\mu_{k,i}=\frac{1}{\mathrm{N}_k}\sum_{\mathrm{j}=1}^{\mathrm{N}_k}x_{j,i}$$

其中，N_k为类别k下的样本数，$\mathrm{x}_{\mathrm{j,i}}$为第 j 个属于第$k$个类别的特征值$x_i$。

5）分类角色，计算每个类别k的后验概率 $\mathrm{P}(\mathrm{C}_k|\mathrm{X})$

$$\mathrm{P}(\mathrm{C}_k|\mathrm{X})$$

将样本X分配到最大后验概率的类别

$$\mathrm{Class(X)} = \arg\max_{C_k} \mathrm{P}(C_k) \prod_{i=1}^{d} \mathrm{P}(x_i|C_k)$$

朴素贝叶斯分类是一种基于贝叶斯定理的概率分类方法，它假设特征之间是条件独立的，即每个特征对分类结果的影响是独立的。模型通过计算各个特征在不同类别下的概率分布，并结合先验概率，来预测样本属于某类别的概率。

（3）度量指标

精确率 Precision：预测为正类的样本中，实际为正类的比例。

$$\mathrm{Precision} = \frac{TP}{TP + FP}$$

召回率 Recall：实际为正类的样本中，预测为正类的比例。

$$\mathrm{Recall} = \frac{TP}{TP + FN}$$

F1-Score：精确率和召回率的调和平均数。

$$\mathrm{F1-Score} = 2 \times \frac{\mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

（4）程序清单（包含详细求解步骤）

1.数据集准备与探索

输出数据集前五行。

```
数据量为: 150
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
```

各列的含义如下：

Sepal_Length：花萼长度，即花萼的最长部分，单位是厘米。

Sepal_Width：花萼宽度，即花萼的最宽部分，单位是厘米。

Petal_Length：花瓣长度，即花瓣的最长部分，单位是厘米。

Petal_Width：花瓣宽度，即花瓣的最宽部分，单位是厘米。

而标签列 Species 中的类别有：

Iris-setosa：山鸢尾

Iris-virginica：维吉尼亚鸢尾

Iris-versicolor：变色鸢尾

2.预处理

1）空值处理

先统计空值

```
空值数量如下:
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

可见不存在空值，所以无需额外处理。

2）归一化

首先统计各列的数据分布，看看每一列各自以及相互之间的数据差异。如果某一列的数据取值之间非常接近（例如一个极端情况是整一列的取值都是 1），那么可能这一列对于建模来说是没有意义的，可以去掉。同时，分位数也能帮我们判断当前列数据中是否存在离群点，可以考虑去掉离群点。此外，如果列和列之间取值范围差得较大（比如一列的取值范围是 0-1，另一列是 100-200）就需要进行归一化/标准化操作，量纲差异太大会导致模型学习过程中对特征重要性的判别出现误差。

这里选择不对原始数据进行归一化或者标准化，因为各列之间的量纲接近。

各列数据数值分布为

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

3）处理“品种 species”列

将不是 virginica 的映射为 0，将 virginica 的映射为 1。

原始为

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

映射后为

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	1
146	6.3	2.5	5.0	1.9	1
147	6.5	3.0	5.2	2.0	1
148	6.2	3.4	5.4	2.3	1
149	5.9	3.0	5.1	1.8	1

[150 rows x 5 columns]

统计数量

```
非virginica的数量:  100
virginica的数量:  50
```

4）划分数据集

划分 70%的样本作为训练数据集，30%的样本作为测试数据集

```
原始数据量150
训练集数据量105
测试集数据量45
```

将所有数据整合成 x-y 的形式（y 就是之前置于尾部的 species）

```
X矩阵为
[[5.5 2.4 3.7 1. ]
 [6.3 2.8 5.1 1.5]
 [6.4 3.1 5.5 1.8]
 [6.6 3.  4.4 1.4]
 [7.2 3.6 6.1 2.5]
 [5.7 2.9 4.2 1.3]
 [7.6 3.  6.6 2.1]
 [5.6 3.  4.5 1.5]
 [5.1 3.5 1.4 0.2]
 [7.7 2.8 6.7 2. ]
 [5.8 2.7 4.1 1. ]
 [5.2 3.4 1.4 0.2]
 [5.  3.5 1.3 0.3]
 [5.1 3.8 1.9 0.4]
 [5.  2.  3.5 1. ]
 [6.3 2.7 4.9 1.8]
```

```
y矩阵为
[[0]
 [1]
 [1]
 [0]
 [1]
 [0]
 [1]
 [0]
 [0]
 [1]
 [0]
 [0]
 [0]
 [0]
 [0]
 [1]
```

3. 逻辑回归模型的梯度下降参数求解

错误记录：由于在定义 theta 时，定义成一个一维的零数组，而不是列向量。因此，矩阵乘法时就会出现维度不匹配的错误。

```
    theta = theta + (alpha / m) * (X.T @ (y - sigmoid(X @ theta)))
            ~~~~~~^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
ValueError: operands could not be broadcast together with shapes (5,) (5,105)
```

下面是错误代码

```
# 初始化参数
theta = np.zeros((X.shape[1])).T  # 创建一个列向量
```

应该改为

```
# 初始化参数
theta = np.zeros((X.shape[1],1))  # 创建一个列向量
```

但是刚生成 theta 时，为 5*1 的向量，当运算完后得到的 theta 为 5*105.

```
theta的形状为(5, 1)
theta的形状为(5, 105)
```

检查发现：打印完整 X、Y 形状的结果如下

```
theta的形状为(5, 1)
y的形状为(105, 1)
X的形状为(105, 5)
theta的形状为(5, 105)
```

于是使用通过 numpy 的广播和布尔索引功能自动进行逐元素比较和计数因为 numpy 的布尔数组可以直接用于逐元素比较，它会生成一个布尔值数组，表示每个元素的匹配情况。

代码如下。

```
# 将预测值转换为类别标签 (0 或 1)
pred_class = (pred >= 0.5).astype(int)
# 计算准确率
correct_num = np.sum(pred_class.flatten() == y_test)  # 逐元素比较并求和
```

最后得到的 45 个预测概率如下，将概率高于 0.5 的处理为 1，其余为 0

最后得到预测的 01 序列.

```
预测概率为
[0.41595473 0.00792698 0.88203174 0.41895109 0.36344209 0.01114369
 0.22909819 0.55115323 0.50527238 0.27759524 0.54480488 0.01575982
 0.00664436 0.0153354  0.00962996 0.37439148 0.7793172  0.3196367
 0.44904692 0.77678952 0.01982443 0.55424945 0.01641284 0.76283774
 0.55909008 0.62888416 0.76048037 0.75077471 0.0183988  0.01981798
 0.00830466 0.00452481 0.24081806 0.01613576 0.01751676 0.6496227
 0.30644767 0.0102239  0.00981039 0.00624254 0.71919651 0.35499304
 0.32452807 0.00631318 0.00774845]
真值序列为
[[0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 1
  0 0 0 0 1 0 0 0 0]]
```

对比真值和预测序列，有一个预测错误。

```
预测序列为
[[0 0 1 0 0 0 0 1 1 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 1
  0 0 0 0 1 0 0 0 0]]
真值序列为
[[0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 1
  0 0 0 0 1 0 0 0 0]]
```

得到实验结果为

```
正确预测个数为 44 个，总个数有 45 个
正确率为 0.9777777777777777
TP为13
FP为1
FN为0
TN为31
Precision:  0.9285714285714286
Recall:     1.0
F1-score:   0.962962962962963
accuracy:   0.9777777777777777
```

手算结果如下，与程序输出一致。

因为在实验结果中，FP=1，TP=13，TN=31，FN=0

则有：

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{13}{13+1} = \frac{13}{14} \approx 0.97778$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{13}{13+0} = 1.0$$

$$\text{F1} - \text{Score} = 2 \times \frac{\frac{13}{14} \times 1}{\frac{13}{14} + 1} \approx 0.96296$$

4、朴素贝叶斯参数统计

在梯度下降的代码基础上，这一部分比较简单，遇到的错误也比较少。

在求取 X 矩阵的特征均值时，按 V 类和非 V 类来统计，但是报错如下。

```
    mean_v = X[:, 1:][y == 1].mean(axis=0)
             ~~~~~~~~^^^^^^^^
IndexError: boolean index did not match indexed array along axis 1; size of axis is 4 but size of corresponding boolean axis is 1
```

是因为 y 的形状和 X 的不匹配，需要将其展平为（105，）

```
y = y.flatten()
# 计算每个特征的均值和标准差
mean_v = X[:, 1:][y == 1].mean(axis=0)
```

最终得出结果，对比真值和预测序列，有 4 个预测错误。

```
原始真值序列为
[0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1 0
 0 0 0 1 0 0 0 0]
预测序列为
[0 0 1 0 1 0 0 1 0 0 1 0 0 0 0 1 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1 1
 0 0 0 1 0 1 0 0]
```

实验结果如下

```
正确预测个数为 41 个，总个数有 45 个
正确率为 0.9111111111111111
TP为13
FP为4
FN为0
TN为28
Precision:  0.7647058823529411
Recall:     1.0
F1-score:   0.8666666666666666
accuracy:   0.9111111111111111
```

手算结果为

FP=4，TP=13，TN=28，FN=0

则有：

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{13}{13+4} = \frac{13}{17} \approx 0.76471$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{13}{13+0} = 1.0$$

$$\text{F1} - \text{Score} = 2 \times \frac{\frac{13}{17} \times 1}{\frac{13}{17} + 1} \approx 0.86667$$

与代码输出一致。

（5）展示与讨论实验结果

梯度下降法求得的结果

```
正确预测个数为 44 个，总个数有 45 个
正确率为 0.9777777777777777
TP为13
FP为1
FN为0
TN为31
Precision:  0.9285714285714286
Recall:     1.0
F1-score:   0.962962962962963
accuracy:   0.9777777777777777
```

CSDN @余蓝

朴素贝叶斯求得的结果

```
正确预测个数为 41 个，总个数有 45 个
正确率为 0.9111111111111111
TP为13
FP为4
FN为0
TN为28
Precision:  0.7647058823529411
Recall:     1.0
F1-score:   0.8666666666666666
accuracy:   0.9111111111111111
```

CSDN @小蓝

可见，梯度下降法的准确率非常高，达到 98%！它仅仅错了 1 个样本，其他的都命中了！正样本的精确度拉高到了 93%，召回率仍旧满分，F1 分数也飙到了 96%。可以说，它在预测 Iris-Virginica 时更加“稳准狠”了！

朴素贝叶斯成功预测了 41 个样本正确分类，整体准确率约为 91%。而且，模型在判断为正样本（Iris-Virginica）时，精确度达到了 76.5%，召回率则是满分的 100%。朴素贝叶斯的 F1 分数也达到了约 87%，非常优秀。

这两种方法的召回率（Recall）均为 1.0，表明它们都对正类样本有较好的识别能力，表现都很好，梯度下降略占优势。

源代码

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# 读取 csv 文件数据
# path = './iris/iris.data'
path = 'iris.data'
# data = pd.read_csv(path)
# 改进：强制指定列名（不管文件是否有表头）
columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
data = pd.read_csv(path, header=None, names=columns)

# 验证列名是否正确
print(" 数据列名：", data.columns.tolist())    # 应该输出 ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
print("类别列唯一值：", data['species'].unique())  #查看类别名称格式
print('数据量为：', len(data))
print(data.head())
print(data)
print('空值数量如下：')
print(data.isnull().sum())
print("各列数据数值分布为")
print(data.describe())
```

```
# 定义类别标签为二分类：virginica 为 1，其他类别为 0
data['species'] = data['species'].apply(lambda x: 1 if x == 'Iris-virginica' else 0)
print(data)

# 统计 species 列中 0 和 1 的数量
counts = data['species'].value_counts()
print("非 virginica 的数量：", counts[0])
print("virginica 的数量：", counts[1])
print("原始数据量{}".format(len(data)))

# 划分样本，7：3
train_data, test_data = train_test_split(data, test_size=0.3, random_state=42)

# print("训练集数据量{}".format((len(train_data))))
# print("测试集数据量{}".format((len(test_data))))

col = train_data.shape[1]  # 求出表格的总列数 col
X = train_data.iloc[:, 0:col - 1].values
y = train_data.iloc[:, col - 1:col].values
X_test = test_data.iloc[:, 0:col - 1].values
y_test = test_data.iloc[:, col - 1:col].values

# m 为训练集的行数
m = X.shape[0]
# n 为测试集行数
n = X_test.shape[0]

# print("X 矩阵为")
# print(X)
# print("\ny 矩阵为")
# print(y)

'''
#将 X,y 分到的数据值转为矩阵类型，便于后续的计算
X = np.matrix(X.values)
y=np.matrix(y.values)
X_test = np.matrix(X_test.values)
y_test=np.matrix(y_test.values)
'''
'''
print("X 矩阵为")
#print(X)
print("\ny 矩阵为")
```

```
#print(y)
'''

# 转置
y = y.T
y_test = y_test.T

# 梯度下降
# 添加偏置项
X = np.c_[np.ones(m), X]   # 第一列加 1
X_test = np.c_[np.ones(n), X_test]
# 初始化参数
# theta = np.zeros((X.shape[1], 1))   # 创建一个列向量
theta = (np.matrix([0, 0, 0, 0, 0])).T
# print("theta 的形状为{}".format(theta.shape))
alpha = 0.01
iter = 1000


# 定义 Sigmoid 函数、损失函数和梯度下降
def sigmoid(z):
    return 1 / (1 + np.exp(-z))


def compute_cost(X, y, theta):
    m_2 = len(y)
    predictions = sigmoid(X @ theta)
    cost = -1 / m_2 * np.sum(y * np.log(predictions) + (1 - y) * np.log(1 - predictions))
    return cost


y = y.reshape(-1, 1)


# print("y 的形状为{}".format(y.shape))
# print("X 的形状为{}".format(X.shape))

# 梯度上升函数
def gradient_boost(theta, alpha):
    for i in range(iter):
        theta = theta + (alpha / m) * (X.T @ (y - sigmoid(X @ theta)))
    return theta
```

```
# 求解θ
theta = gradient_boost(theta, alpha)
# print("theta 的形状为{}".format(theta.shape))
# print("X_test 的形状为{}".format(X_test.shape))
# 计算准确率
pred = sigmoid(X_test @ theta)
correct_num = 0

# 将 pred 和 y_test 展平为一维数组
# pred = pred.reshape(-1)
# print("*pred 的形状为{}".format(pred.shape))
# print(pred)
# print("*y_test 的形状为{}\n".format(y_test.shape))

# 将预测值转换为类别标签（0 或 1）
pred_class = (pred >= 0.5).astype(int)
# print("预测序列为\n{}".format(pred_class))
# print("真值序列为\n{}".format(y_test))

# 计算准确率
correct_num = np.sum(pred_class.flatten() == y_test)  # 逐元素比较并求和
print("正确预测个数为 {} 个，总个数有 {} 个".format(correct_num, n))
rate = float(correct_num) / n  # 准确率 = 正确分类的数量 / 总样本数量
print("正确率为 {}".format(rate))

# 将预测结果与实际结果进行比较
TP = np.sum((pred_class.flatten() == 1) & (y_test == 1))
FP = np.sum((pred_class.flatten() == 1) & (y_test == 0))
FN = np.sum((pred_class.flatten() == 0) & (y_test == 1))
TN = np.sum((pred_class.flatten() == 0) & (y_test == 0))
print("TP 为{}".format(TP))
print("FP 为{}".format(FP))
print("FN 为{}".format(FN))
print("TN 为{}".format(TN))

# 计算 Precision 和 Recall,以及 accuarcy
precision = TP / (TP + FP)
recall = TP / (TP + FN)
accuracy = (TP + TN) / (TP + TN + FP + FN)

# 计算 F1-score
f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

print("Precision: ", precision)
```

```
print("Recall:       ", recall)
print("F1-score:    ", f1_score)
print("accuracy:    ", accuracy)

print("---"*20)

#朴素贝叶斯
# 计算每个类别的先验概率
v_prior = np.mean(y)   # Virginica 的先验概率
nv_prior = 1 - v_prior   # 非 Virginica 的先验概率
y = y.flatten()
# 计算每个特征的均值和标准差
mean_v = X[:, 1:][y == 1].mean(axis=0)
std_v = X[:, 1:][y == 1].std(axis=0)
mean_nv = X[:, 1:][y == 0].mean(axis=0)
std_nv = X[:, 1:][y == 0].std(axis=0)

#高斯分布概率密度函数
def gaussian_prob(x, mean, std):
    return (1 / (np.sqrt(2 * np.pi) * std)) * np.exp(-0.5 * ((x - mean) / std) ** 2)

# 预测函数
def predict(X):
    predictions = []
    for x in X:
        # 计算每个特征在 Virginica 类别下的条件概率
        prob_v = np.prod(gaussian_prob(x, mean_v, std_v)) * v_prior
        # 计算每个特征在非 Virginica 类别下的条件概率
        prob_nv = np.prod(gaussian_prob(x, mean_nv, std_nv)) * nv_prior
        # 比较两个类别的概率，取概率大的类别
        predictions.append(1 if prob_v > prob_nv else 0)
    return np.array(predictions)

# 在测试集上预测
pred = predict(X_test[:, 1:])
y_test = y_test.flatten()
print("原始真值序列为")
print(y_test)
print("预测序列为")
print(pred)

# 计算准确率
correct_num = np.sum(pred == y_test)   # 逐元素比较并求和
print("正确预测个数为 {} 个，总个数有 {} 个".format(correct_num,n))
```

```
rate = float(correct_num) / n   # 准确率 = 正确分类的数量 / 总样本数量
print("正确率为 {}".format(rate))

# 将预测结果与实际结果进行比较
TP = np.sum((pred == 1) & (y_test == 1))
FP = np.sum((pred == 1) & (y_test == 0))
FN = np.sum((pred == 0) & (y_test == 1))
TN = np.sum((pred == 0) & (y_test == 0))
print("TP 为{}".format(TP))
print("FP 为{}".format(FP))
print("FN 为{}".format(FN))
print("TN 为{}".format(TN))

# 计算 Precision 和 Recall,以及 accuarcy
precision = TP / (TP + FP)
recall = TP / (TP + FN)
accuracy = (TP+TN)/(TP+TN+FP+FN)
# 计算 F1-score
f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
print("Precision: ", precision)
print("Recall:       ", recall)
print("F1-score:    ", f1_score)
print("accuracy:    ",accuracy)
```